

PERSONAL THEME

AI Agents for Education and Reliable Workflows

I am interested in building AI agents that do more than answer questions. The agents I want to build should understand real workflows, call tools, ask for human approval when needed, and help people make better decisions with less repetitive work.

My background connects physics, software engineering, machine learning systems, and education technology. In research, I studied LLM inference and compiler systems, where backend choices, floating-point precision, and evaluation pipelines can affect model behavior. In education, I saw a different problem: teachers already have digital platforms, but still spend too much time switching pages, checking student progress, assigning practice, and following up manually.

Working question

How can AI agents become reliable workflow partners for education?

I also actively use AI coding tools such as Codex and Claude Code in my engineering workflow. I use them to read codebases, draft implementation plans, debug, compare alternatives, and write documentation. Using these tools every day shaped how I think about agents: a useful agent needs context, tools, memory, constraints, and human control.

Human workflow -> Agent -> Tools -> Data -> Human decision

NEW ORIENTAL

From Traditional OJ to Agent-Assisted Teaching

During my Teaching Fellow (Management Trainee) experience at Suzhou New Oriental Training School, I coached students for informatics learning, including DFS/BFS, dynamic programming, arrays, and linked lists.

Working close to daily teaching operations helped me notice a limitation of traditional OJ systems. They are good at judging submissions, but weak at supporting what teachers need after judging.

- Who is falling behind?
- Which problems are blocking the class?
- Which student needs easier practice before moving on?
- Which plan should be adjusted this week?

I started thinking about the OJ platform not only as a judge, but as a teaching workflow system. My approach was to wrap core CRUD services into MCP tools, including problem management, learning plans, submissions, student progress, and recommendation-related data.

Once these services become tools, an agent can help teachers query progress, compare students, adjust plans, and generate personalized practice suggestions through natural language. The value is not in replacing teachers; it is in reducing operational friction so teachers can spend more time on judgment, guidance, and communication.

Teacher -> Natural language request -> MCP tools -> Agent summary -> Teacher confirmation

LIVE EDTECH SYSTEM

GESP Intelligent Practice Platform

Live platform: <https://gesp.growplan.top/>

I built and developed the GESP practice platform, an online programming practice and growth plan system used by about 1,000 students and teachers. The platform supports OJ programming practice, learning plans, student progress tracking, and teacher-side management.

The first version solved the basic infrastructure problem: students need a place to practice, submit code, and track progress; teachers need a way to assign plans and see completion. The more interesting problem is what comes next. A fixed plan is often too rigid. Different students fail for different reasons, and teachers cannot manually inspect every submission history all the time.

The direction I explored is to upgrade the platform into an adaptive learning loop:

Practice -> Judge -> Track Progress

Identify Weakness -> Recommend Next Task -> Teacher Follow-up

In this design, OJ submissions and learning-plan progress become signals for an agent. The agent can analyze verdicts, unfinished tasks, repeated wrong attempts, weak topics, and practice duration, then recommend the next set of tasks. Teachers still keep control, but the system gives them better context and faster follow-up options.

- Full-stack education platform with real student and teacher usage
- A path from CRUD services to agent-driven learning support

HUMAN-IN-THE-LOOP AUTOMATION

A Controllable WeCom Agent

GitHub: <https://github.com/Notborntodie/wecom-agent/>

At New Oriental, I built a WeCom agent used internally. The goal was not to create a fully autonomous bot. In a teaching and management environment, uncontrolled automation is risky. A wrong message can affect students, parents, teachers, or internal operations.

So I designed the agent around a controllable workflow:

Message sensing -> LLM reply planning -> Human approval -> Safe execution

The agent turns a closed desktop communication environment into a semi-automated workflow. It reads relevant message context, drafts possible replies or actions with an LLM, asks for confirmation, and only executes after approval. This project strengthened my belief that safety is not an extra feature for applied agents; it is part of the architecture.

This also mirrors how I work with Codex and Claude Code. I use them to inspect code, propose diffs, find edge cases, and document decisions, while keeping the final responsibility and system judgment human-led.

- Practical agent engineering in an internal work environment
- Human approval and safe execution as first-class design choices

CONFIDENTIAL UGA RESEARCH

LLM Systems, Compiler Backends, and Reliability

Role: Research Assistant, The University of Georgia, Jun 2024 - Jan 2025

Note: confidential research experience; project details and data are not public.

At the University of Georgia, I worked on confidential research related to deep learning compiler systems and LLM inference. Although I cannot disclose project details, this experience gave me a systems-level view of AI.

I worked with LLM inference and compilation frameworks such as llama.cpp and MLC-LLM. I explored backend support, including Vulkan, and enabled evaluation workflows with lm_eval, a widely used framework for measuring LLM accuracy across tasks.

The most valuable part was learning to think about AI reliability below the application layer. In LLM systems, a model output is not only determined by a prompt. It can also be affected by inference kernels, compiler optimizations, backend differences, floating-point precision, and evaluation setup.

Small numerical differences in logits may accumulate and eventually lead to different generated outputs, especially in autoregressive decoding. This experience made me more careful about agent systems: useful agents need reliable inference, careful evaluation, transparent tool use, and reproducible behavior where possible.

Model -> Inference runtime -> Compiler/backend -> Hardware -> Evaluation

PRODUCTION DEBUGGING + FUTURE DIRECTION

Learning from Real Failures, Then Building Better Agents

Publication: Demystifying Developers' Fight Against Complexity: A Comprehensive Study of Live Debugging Activities in Production Cloud Systems. SoCC 2024.

In my remote research experience with Purdue University, I worked on a project studying how developers debug complex production cloud systems. My main contribution was collecting and organizing public real-world debugging cases from distributed systems and databases. This supported the analysis behind the SoCC 2024 publication.

This project taught me how engineers reason under uncertainty. In production debugging, people rarely know the answer at the beginning. They collect evidence, form hypotheses, inspect logs and traces, test assumptions, and gradually narrow down the problem.

A useful agent should behave more like a careful engineer than a text generator. It should gather context, call tools, verify assumptions, expose uncertainty, and keep humans in control when decisions matter.

This is why the Red Bird MPhil is attractive to me. I want to work in an environment where project-based research is connected with real systems and real users. My goal is to build AI agents for education that are practical, controllable, and technically reliable.

- GitHub: <https://github.com/Notborntodie>
- GESP: <https://gesp.growplan.top/>
- WeCom Agent: <https://github.com/Notborntodie/wecom-agent/>
- SoCC 2024 Paper: <https://yonglezh-purdue.github.io/files/socc24-live-debugging.pdf>